

機器人學 Project2

313512009 施丞蔚 william0726.tai@gmail.com

一、介面說明（開發平台、如何執行...）

- 執行環境：Windows 11
- 開發工具：MATLAB R2023a
- 程式語言：MATLAB
- 如何執行：直接透過 MATLAB 內建的編譯器執行，或在 MATLAB 指令視窗輸入 `main` 執行主程式。

二、程式架構說明（程式運行流程、核心程式碼說明...）

- 程式運行流程：
 1. 參數定義：初始化起點與終點、過渡時間及取樣率。
 2. 正運動學與逆運動學：分別使用函式 `FK.m` 和 `IK.m` 計算正運動學與逆運動學。
 3. 軌跡規劃：主程式 (`main.m`) 計算關節與笛卡兒空間的軌跡，同時考慮速度與加速度限制。
 4. 結果視覺化：以曲線及 3D 路徑圖展示結果。
- 核心程式碼說明：
 1. `FK.m`：
計算正運動學，根據給定的關節角度得出笛卡兒位置與姿態。
 2. `IK.m`：
解逆運動學，將笛卡兒位置與姿態轉換為關節角度。
 3. `exceed_range.m`：
檢查關節角度是否超出允許範圍。
 4. `EA2RM.m` 與 `RM2EA.m`：
在歐拉角與旋轉矩陣之間進行轉換，用於運動學計算
 5. `main.m`：
實現關節與笛卡兒運動的軌跡規劃，以及結果視覺化。

三、數學運算說明

1. Denavit-Hartenberg (DH) 參數法：

- DH 參數法是一種用於描述多關節機械臂各連桿之間相對位置的標準化方法。每個關節由四個參數來描述：d (前臂長度)、a (連桿長度)、alpha (連桿扭轉角) 和 theta (關節角)。
- 齊次變換矩陣 A_i 的計算基於 DH 參數：

$$A_i = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \cos(\alpha) & \sin(\theta) \sin(\alpha) & a \cos(\theta) \\ \sin(\theta) & \cos(\theta) \cos(\alpha) & -\cos(\theta) \sin(\alpha) & a \sin(\theta) \\ 0 & \sin(\alpha) & \cos(\alpha) & d \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- 將所有六個關節的變換矩陣相乘，得到最終的末端執行器的齊次變換矩陣 T。

2. 正向運動學推導：

- 齊次變換矩陣 T：
 - 繼續相乘每個連桿的變換矩陣，最終得到一個總變換矩陣 T，描述從基座到末端執行器之間的位姿變換。
 - T 的形式為：

$$T = \begin{bmatrix} n_x & o_x & a_x & p_x \\ n_y & o_y & a_y & p_y \\ n_z & o_z & a_z & p_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

其中，n、o、a 分別為旋轉矩陣的三列向量，p 為位置向量。

3. 計算 Z-Y-Z 歐拉角：

- 歐拉角的計算是根據 T 中的旋轉矩陣部分來確定末端執行器的姿態。
- 使用 Z-Y-Z 旋轉順序，對於旋轉矩陣中描述的各列向量，我們使用以下方法來計算：
 - 第一個旋轉角 (ψ)：計算繞 Z 軸的旋轉，使用 $\text{atan2}(T[1, 2], T[0, 2])$ 。
 - 第二個旋轉角 (θ)：計算繞 Y 軸的旋轉，使用 $\text{atan2}(\sqrt{T[0, 2]^2 + T[1, 2]^2}, T[2, 2])$ 。
 - 第三個旋轉角 (ϕ)：計算繞 Z 軸的旋轉，使用 $\text{atan2}(T[2, 1], -T[2, 0])$ 。
- 最後將這些角度從弧度轉換為角度。

逆向運動學(Project1_2.py):

1. 齊次變換矩陣 A 的定義：

$$A = \begin{bmatrix} nx & ox & ax & Px \\ ny & oy & ay & Py \\ nz & oz & az & Pz \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- nx, ny, nz 代表 n 向量的分量，即機器人末端在空間中的 x, y, z 方向上的法向量。
- ox, oy, oz 代表 o 向量的分量，即機器人末端在空間中的 x, y, z 方向上的橫向向量。
- ax, ay, az 代表 a 向量的分量，即機器人末端在空間中的 x, y, z 方向上的姿態向量。
- Px, Py, Pz 代表機器人末端在空間中的位置。

2. 計算各個關節角度的步驟：

- **cta1 (第一關節)的推導：**

cta1 通常是基於末端位置 Px 和 Py 計算出來，代表機器人基座旋轉的角度。使用的公式為：

$$cta1 = \text{atan2}(Py, Px)$$

這個公式可以計算出機器人的基座如何旋轉來對準末端的目標位置， atan2 函數能夠確保計算結果位於正確的象限中。

- **cta5 (第五關節)的推導：**
- cta5 的推導主要基於末端的姿態向量 ax 和 ay ，以及第一關節的旋轉結果：

$$cta5 = \text{acos}(-\sin(cta1) \cdot ax + \cos(cta1) \cdot ay)$$

這個公式用於計算末端姿態在水平方向上的偏移。由於第五關節通常是控制機器人腕部的姿態，因此它與姿態向量的投影有關。

- **cta6 (第六關節)的推導：**
- cta6 代表機器人末端旋轉的方向，主要由 ox, oy, nx, ny 和 $cta1$ 計算得出：

$$cta6 = \text{atan2}(-\sin(cta1) \cdot ox + \cos(cta1) \cdot oy, \sin(cta1) \cdot nx - \cos(cta1) \cdot ny)$$

此角度表示機器人末端的橫向旋轉，與姿態矩陣中的元素有關。

- **cta3 (第三關節)的推導：**

- cta3 通常是基於餘弦定理來計算，這裡使用了機器人臂長參數 a_1, a_2, a_3 和末端位置 P_x, P_y, P_z 。餘弦定理公式如下：

$$\cos(\text{cta3}) = \frac{P_x^2 + P_y^2 + P_z^2 - 2a_1(\cos(\text{cta1}) \cdot P_x + \sin(\text{cta1}) \cdot P_y) + a_1^2 - a_3^2 - a_2^2}{2a_2a_3}$$

cta3 可以通過 acos 來計算，這表示第三個關節的角度，它控制機器人的肘部位置。

- **cta2 (第二關節)的推導：**

cta2 的推導是基於 cta3、 P_x, P_y, P_z 以及臂長 a_2 和 a_3 。公式為：

$$\cos(\text{cta2}) = \frac{a_3 \cos(\text{cta3})(\cos(\text{cta1})P_x + \sin(\text{cta1})P_y - a_1) - a_3 \sin(\text{cta3})P_z + a_2(\cos(\text{cta1})P_x + \sin(\text{cta1})P_y - a_1)}{(\cos(\text{cta1})P_x + \sin(\text{cta1})P_y - a_1)^2 + P_z^2}$$

cta2 使用 acos 計算，代表機器人肩部的角度。

- **cta4 (第四關節)的推導：**

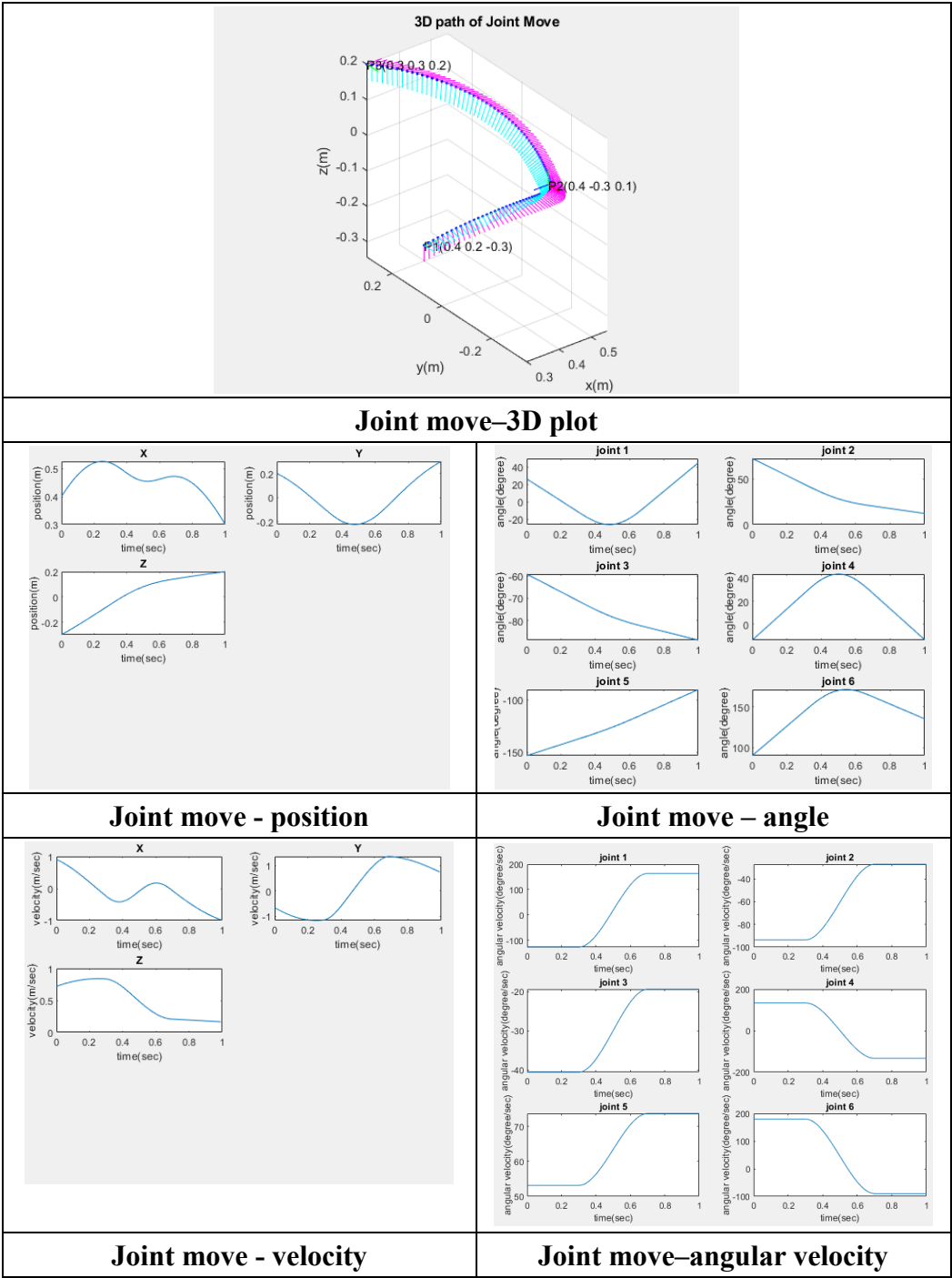
cta4 通常是基於 cta1、cta2、cta3 以及末端姿態向量來計算，使用 atan2 函數：

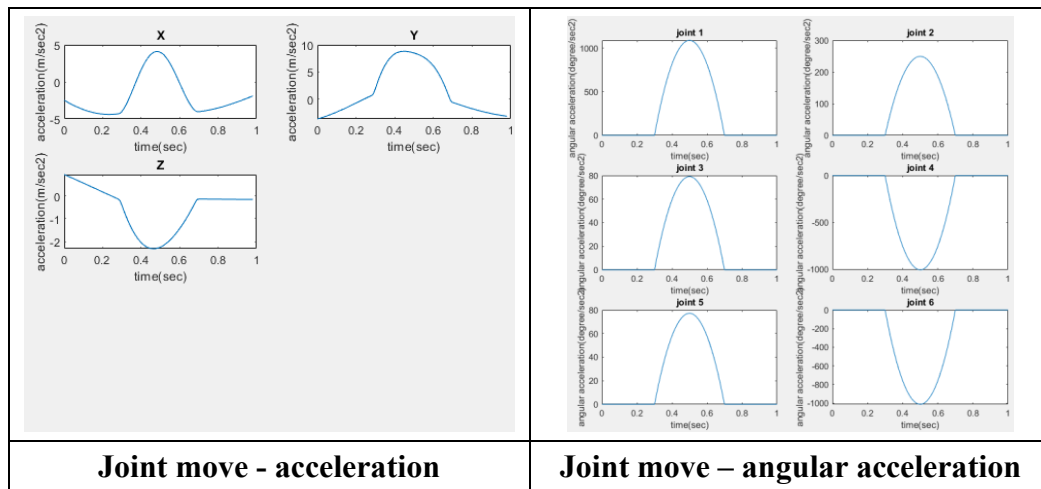
$$\text{cta4} = \text{atan2}(-\cos(\text{cta1})\sin(\text{cta2} + \text{cta3}) \cdot a_x - \sin(\text{cta1})\sin(\text{cta2} + \text{cta3}) \cdot a_y - \cos(\text{cta2} + \text{cta3}) \cdot a_z, \cos(\text{cta1})\cos(\text{cta2} + \text{cta3}) \cdot a_x + \sin(\text{cta1})\cos(\text{cta2} + \text{cta3}) \cdot a_y - \sin(\text{cta2} + \text{cta3}) \cdot a_z)$$

這個角度控制機器人手腕的旋轉方向。

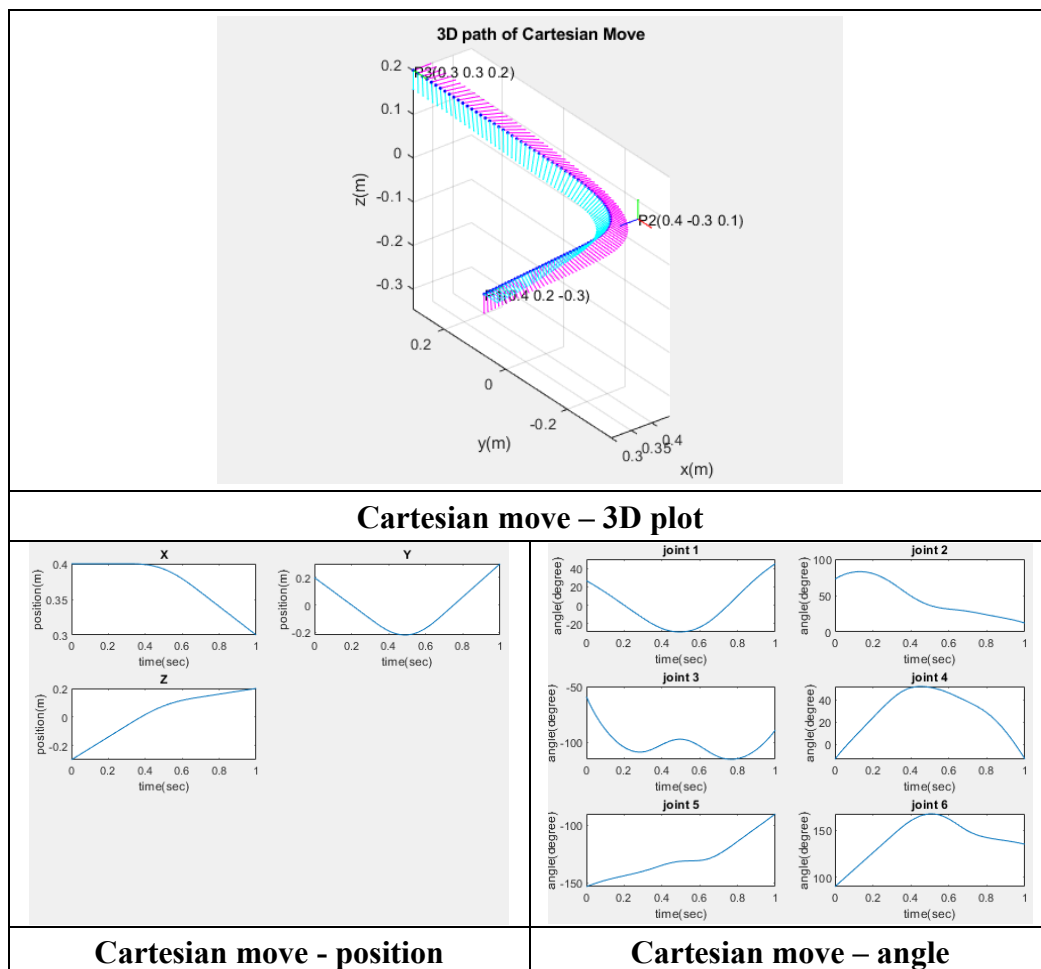
四、軌跡規劃曲線圖結果

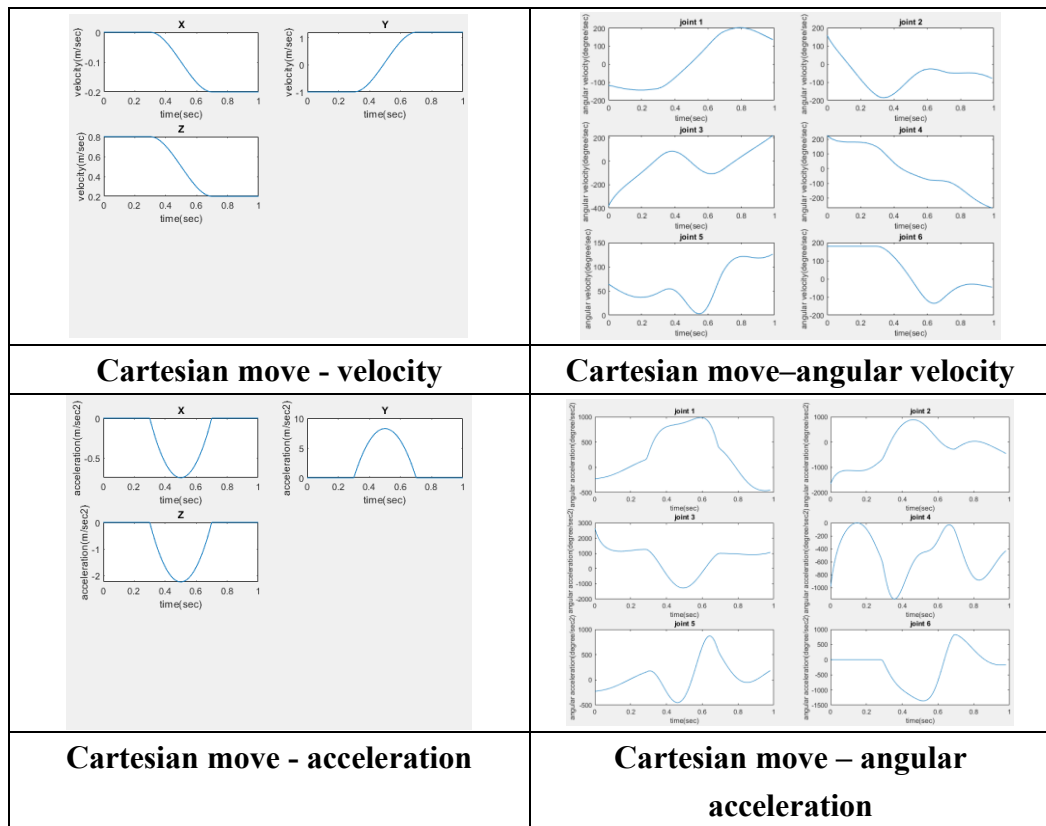
1. Joint move





2. Cartesian move





五、加分題：討論兩種軌跡規劃的優缺點、遇到奇異點如何處理。

1. Joint move

優點：

- 計算簡單，直接基於關節角度進行插值。
- 確保關節的平滑運動，避免奇異點問題。

缺點：

- 無法直接控制末端執行器在笛卡兒空間的路徑，可能出現非期望的空間運動。

2. Cartesian move

優點：

- 可以精確控制末端執行器的空間路徑，適用於要求高的應用場景。

缺點：

- 需要逆運動學計算，可能導致多解或奇異點問題。
- 關節可能產生大幅度運動。

奇異點處理

- 避免奇異點，在規劃軌跡時，避免機器人到達奇異點附近的姿態。
- 數值阻尼法，在雅可比矩陣中加入阻尼項以緩解奇異點問題。
- 重規劃路徑，在奇異點附近改變運動路徑，避開奇異點。

六、參考資料

機器人學講義-楊谷洋老師